ACADEMIC SESSION 2026–2027

LAB PRACTICAL FILE

DATA STRUCTURES

SUBMITTED TO	SUBMITTED BY
Mr. Sachin Sharma	Name: Priya
Faculty / Assistant Professor	Roll No.: 26MCAP101N019
School of Computer Applications	Semester: 1
MRIIRS	Course: MCA

A carefully organised set of Python 3 practicals covering arrays, matrices, stacks and queues. Each practical includes an aim, core concept, program, illustrative run and complexity note.

INDEX

No.	Practical
1	Insert an element into an array
2	Delete an element from an array
3	Multiply two matrices
4	Sum diagonal and non-diagonal elements
5	Marks and percentage for five students
6A	Stack operations
6B	Reverse a string using a stack
6C	Balanced-parentheses matching
7A	Linear queue operations
7B	Circular queue operations

Conventions: Array positions entered by the user are 1-based; Python indices are 0-based. Matrix and mark inputs are checked for valid dimensions/ranges.

PRACTICAL 1 | Insert an element into an array

Aim

Insert a value at a specified 1-based position while preserving the order of elements.

Concept and method

Positions range from 1 through n+1. Convert to a Python index with position - 1; list insertion shifts later values.

Python 3 program

```
def read_int(prompt):
    while True:
        try: return int(input(prompt))
        except ValueError: print("Enter a whole number.")
n = read_int("Number of elements: ")
while n < 0:
    print("Size cannot be negative."); n = read_int("Number of elements: ")
arr = [read_int(f"Element {i+1}: ") for i in range(n)]
print("Original array:", arr)
value = read_int("Value to insert: ")
pos = read_int(f"Position (1 to {n+1}): ")
if 1 <= pos <= n+1:
    arr.insert(pos-1, value)
    print("Array after insertion:", arr)
else: print("Invalid position.")
```

Illustrative run

```
Original array: [12, 14, 16, 18, 20]
Value to insert: 50
Position (1 to 6): 3
Array after insertion: [12, 14, 50, 16, 18, 20]
```

Complexity: Worst-case time O(n); list storage O(n).

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 2 | Delete an element from an array

Aim

Remove the first occurrence of a specified value, reporting clearly if it is absent.

Concept and method

Search the array first; remove() deletes only the first matching value. If the value is missing, the array remains unchanged.

Python 3 program

```
def read_int(prompt):
    while True:
        try: return int(input(prompt))
        except ValueError: print("Enter a whole number.")
n = read_int("Number of elements: ")
while n < 0:
    print("Size cannot be negative."); n = read_int("Number of elements: ")
arr = [read_int(f"Element {i+1}: ") for i in range(n)]
print("Original array:", arr)
value = read_int("Value to delete: ")
if value in arr:
    arr.remove(value)
    print("Array after deletion:", arr)
else: print("Value not found; array unchanged:", arr)
```

Illustrative run

```
Original array: [10, 20, 30, 40, 50]
Value to delete: 30
Array after deletion: [10, 20, 40, 50]
```

Complexity: O(n) time for search and shifting; O(1) extra space.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 3 | Multiply two matrices

Aim

Compute A × B when the number of columns of A equals the number of rows of B.

Concept and method

For A(m×n) and B(n×p), C[i][j] = Σ A[i][k]B[k][j]. The result has dimensions m×p.

Python 3 program

```
def read_matrix(rows, cols, label):
    result = []
    print(f"Enter {label}, one row per line:")
    for i in range(rows):
        while True:
            try:
                row = list(map(int, input(f"Row {i+1}: ").split()))
                if len(row) == cols:
                    result.append(row); break
                print(f"Enter exactly {cols} integers.")
            except ValueError: print("Use space-separated integers.")
r1,c1 = map(int,input("Rows and columns of A: ").split())
r2,c2 = map(int,input("Rows and columns of B: ").split())
if min(r1,c1,r2,c2) <= 0: print("Dimensions must be positive.")
elif c1 != r2: print("Incompatible dimensions: columns of A must equal rows of B.")
else:
    A=read_matrix(r1,c1,"A"); B=read_matrix(r2,c2,"B")
    C=[[sum(A[i][k]*B[k][j] for k in range(c1))
        for j in range(c2)] for i in range(r1)]
    print("Product A x B:")
    for row in C: print(*row)
```

Illustrative run

```
A = [[1, 3], [5, 7]]; B = [[2, 4], [6, 8]]
Product A x B:
20 28
52 76
```

Complexity: O(r1 × c1 × c2) time; O(r1 × c2) result storage.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 4 | Diagonal and non-diagonal sums

Aim

Calculate the primary-diagonal and remaining-element sums of a square matrix.

Concept and method

A primary-diagonal entry has matching row and column indices (i == j). All other entries contribute to the non-diagonal sum.

Python 3 program

```
n = int(input("Order of square matrix: "))
if n <= 0:
    print("Order must be positive.")
else:
    matrix=[]
    for i in range(n):
        while True:
            try:
                row=list(map(int,input(f"Row {i+1}: ").split()))
                if len(row)==n: matrix.append(row); break
                print(f"Enter exactly {n} integers.")
            except ValueError: print("Use space-separated integers.")
    diagonal=non_diagonal=0
    for i in range(n):
        for j in range(n):
            if i==j: diagonal += matrix[i][j]
            else: non_diagonal += matrix[i][j]
    print("Primary diagonal sum:",diagonal)
    print("Non-diagonal sum:",non_diagonal)
```

Illustrative run

```
Row 1: 1 2 3
Row 2: 4 5 6
Row 3: 7 8 9
Primary diagonal sum: 15
Non-diagonal sum: 30
```

Complexity: $O(n^2)$ time and $O(n^2)$ storage for the matrix.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 5 | Marks and percentage for five students

Aim

Read three subject marks per student and calculate each total and percentage.

Concept and method

Assume each subject is out of 100, so maximum marks = 300. Percentage = total / 300 × 100.

Python 3 program

```
results=[]
for student in range(1,6):
    marks=[]
    print(f"\nStudent {student}")
    for subject in range(1,4):
        while True:
            try:
                mark=float(input(f"Subject {subject} mark (0-100): "))
                if 0 <= mark <= 100: marks.append(mark); break
                print("Mark must be from 0 to 100.")
            except ValueError: print("Enter a numeric mark.")
    total=sum(marks); results.append((total,total/300*100))
print("\nSUMMARY")
for i,(total,percent) in enumerate(results,1):
    print(f"Student {i}: {total:g}/300, {percent:.2f}%")
```

Illustrative run

```
Student 1 marks: 80, 75, 85
Student 2 marks: ...
(Enter all 15 marks.)
SUMMARY
Student 1: 240/300, 80.00%
Student 2: .../300, ...%
```

Complexity: O(1) per mark; fixed 5 × 3 dataset.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 6A | Stack operations

Aim

Implement push, pop, peek and display using a Python list.

Concept and method

A stack follows LIFO (Last In, First Out). Empty-stack checks prevent underflow.

Python 3 program

```
stack=[]
while True:
    print("\n1 Push  2 Pop  3 Peek  4 Display  5 Exit")
    choice=input("Choice: ").strip()
    if choice=="1":
        value=input("Value: "); stack.append(value); print("Pushed:",value)
    elif choice=="2":
        print("Stack underflow." if not stack else "Popped: "+stack.pop())
    elif choice=="3":
        print("Stack is empty." if not stack else "Top:",stack[-1] if stack else "")
    elif choice=="4": print("Bottom to top:",stack)
    elif choice=="5": break
    else: print("Invalid choice.")
```

Illustrative run

```
Choice: 1; Value: 12  → Pushed: 12
Choice: 1; Value: 30  → Pushed: 30
Choice: 3             → Top: 30
Choice: 2             → Popped: 30
Choice: 4             → Bottom to top: ['12']
```

Complexity: Push/pop/peek O(1) average; display O(n).

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 6B | Reverse a string using a stack

Aim

Reverse input text by pushing its characters and popping them.

Concept and method

LIFO order returns characters in the opposite order from insertion.

Python 3 program

```
text=input("Enter a string: ")
stack=[]
for ch in text: stack.append(ch)
reversed_text=""
while stack: reversed_text += stack.pop()
print("Original:",text)
print("Reversed:",reversed_text)
```

Illustrative run

```
Enter a string: Data Structures
Original: Data Structures
Reversed: serutcurtS ataD
```

Complexity: O(n) time and O(n) extra space.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 6C | Balanced-parentheses matching

Aim

Check correct pairing and nesting of (), [] and {} in an expression.

Concept and method

Push opening brackets. A closing bracket must match the latest opener. Ignore other characters; success requires an empty stack at the end.

Python 3 program

```
expr=input("Enter an expression: ")
stack=[]
pairs={')':'(',']':'[','}':'{'}
valid=True
for ch in expr:
    if ch in pairs.values(): stack.append(ch)
    elif ch in pairs:
        if not stack or stack.pop()!=pairs[ch]:
            valid=False; break
if stack: valid=False
print("Balanced" if valid else "Not balanced")
```

Illustrative run

```
Enter an expression: (a + [b * {c-d}])
Balanced
Enter an expression: ([)]
Not balanced
```

Complexity: O(n) time and O(n) worst-case space.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 7A | Linear queue operations

Aim

Implement enqueue, dequeue, front and display with a fixed capacity.

Concept and method

A queue follows FIFO (First In, First Out). This simple list implementation checks overflow and underflow.

Python 3 program

```
capacity=int(input("Queue capacity: "))
while capacity<=0: capacity=int(input("Enter positive capacity: "))
queue=[]
while True:
    print("\n1 Enqueue  2 Dequeue  3 Front  4 Display  5 Exit")
    choice=input("Choice: ").strip()
    if choice=="1":
        if len(queue)==capacity: print("Queue overflow: full.")
        else:
            value=input("Value: "); queue.append(value); print("Enqueued:",value)
    elif choice=="2":
        if not queue: print("Queue underflow: empty.")
        else: print("Dequeued:",queue.pop(0))
    elif choice=="3": print("Empty" if not queue else "Front:"+str(queue[0]))
    elif choice=="4": print("Front to rear:",queue)
    elif choice=="5": break
    else: print("Invalid choice.")
```

Illustrative run

```
Capacity: 3
Enqueue A; Enqueue B
Front: A
Dequeued: A
Front to rear: ['B']
```

Complexity: Enqueue O(1); dequeue O(n) in a Python list due to shifting.

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

PRACTICAL 7B | Circular queue operations

Aim

Implement a fixed-capacity circular queue using an array and front/rear indices.

Concept and method

The modulo operator wraps indices. Empty: front == -1. Full: (rear + 1) % capacity == front. Vacated cells can be reused.

Python 3 program

```
n=int(input("Queue capacity: "))
while n<=0: n=int(input("Enter positive capacity: "))
q=[None]*n; front=rear=-1
while True:
    print("\n1 Enqueue  2 Dequeue  3 Display  4 Exit")
    choice=input("Choice: ").strip()
    if choice=="1":
        if (rear+1)%n==front: print("Circular queue is full.")
        else:
            value=input("Value: ")
            if front==-1: front=0
            rear=(rear+1)%n; q[rear]=value; print("Enqueued:",value)
    elif choice=="2":
        if front==-1: print("Circular queue is empty.")
        else:
            value=q[front]; q[front]=None; print("Dequeued:",value)
            if front==rear: front=rear=-1
            else: front=(front+1)%n
    elif choice=="3":
        if front==-1: print("Circular queue is empty.")
        else:
            items=[]; i=front
            while True:
                items.append(q[i])
                if i==rear: break
                i=(i+1)%n
            print("Front to rear:",items)
    elif choice=="4": break
    else: print("Invalid choice.")
```

Illustrative run

```
Capacity: 3
Enqueue 10; Enqueue 20; Dequeue → 10
Enqueue 30; Enqueue 40
Front to rear: ['20', '30', '40']
```

Complexity: Enqueue/dequeue O(1); display O(n).

Result: The program demonstrates the stated data-structure operation and its relevant boundary checks.

KEY CONCEPTS AND CHECKS

Array/list: Ordered data; insertion or deletion in the middle shifts later values.

Matrix product: Inner dimensions must match: (m×n)(n×p) gives an m×p matrix.

Stack: LIFO — the most recently added item is removed first.

Queue: FIFO — the earliest added item is removed first. Circular indexing reuses array positions.

Robust input: Validate dimensions, positions, queue capacity and mark ranges before processing.

END OF PRACTICAL FILE